

Lab A,DSP

Prof. Naomi Harte, Dr Hugh O'Dwyer

September 26, 2026

In this lab you will take on a series of tasks relating to audio signals and music. A fundamental knowledge of MATLAB is required to complete these labs. You should take a refresher course online in MATLAB if you feel you need it.

Visit <https://matlabacademy.mathworks.com/details/matlab-onramp/gettingstarted> if you need a course.

Lab 0 was also offered to you to help bring you up to speed. Similar to the refresher MATLAB course, both parts of this lab are arranged in sequential tasks which must be completed before moving onto the subsequent task. You are presented with a MATLAB file for each part of this lab which you should edit to complete what is being asked of you in the Lab. Change the name of this file to replace the word Student with your name.

While you do not need to write a report, we recommend that you make a document to record your work as you go, to aid you in reviewing this material. As a suggestion, you could include all figures you generate, record any observations you make, or responses to questions asked in this handout. You could do this in the form of bulleted points or similar. You need to be familiar with the sort of tasks given to you here for your end of Semester Class MATLAB Assignment. So before you start, think about the best way for you to do this.

The minimum required submission is your code. All code should be appropriately commented. No formal report is required and no marks are assigned for this Lab. However, any material that you submit can later be used during the Class MATLAB Assignment. Thus, if you think you may find your notes helpful, then submit them too. This assignment is formative, and the Clinic is your opportunity to ask questions and get feedback.

Students should refer to Blackboard for the associated deadline for this Lab.

You will need headphones to listen to the signals when necessary. Before listening to any signals, please check your volume to avoid damaging your hearing.

Do NOT resort directly to GenAI tools to write the code in this assignment. If you do, you are entirely missing the point of the exercise. Try each task on your own first. If you find GenAI tools helpful, you can use them to figure out where you are going wrong etc. It is vital you understand any code you use.

Remember you should complete most of this lab in ADVANCE of attending the clinic, using the scheduled clinic to get help with any remaining issues you have.

1 Part 1: Audio Signals

To begin, please open the Matlab script 'labAPart1Student.m' provided to you. Fill in your answer for each task in the blank section under each question in the script.

Task 1 - Defining the attributes of an Audio Signal.

The signal x is given as:

$$x[n] = \sin(2\pi f_0 t[n])$$

x has a sampling frequency f_s of 8000Hz, a fundamental frequency f_0 of 900 Hz, and a duration, d of 3 seconds.

Define these three variables in MATLAB.

Task 2 - Determining the number of samples in an audio signal.

Determine the value of N , the total number of samples in this audio signal.

Task 3 - Define the time vector, $t[n]$.

The corresponding times $t[n]$ are defined as:

$$t[n] = \frac{n}{f_s} \quad \text{for } 0 \leq n \leq N - 1$$

define $t[n]$ as a vector of times when the audio signal is sampled.

Task 4 - Defining $x[t]$.

In Matlab, define the audio signal $x[t]$.

Task 5 - Listen to $x[t]$.

Use the sound function in Matlab to listen to the audio signal $x[n]$.

```
sound(x, fs) %listen to signal x at a sampling frequency, Fs.
```

Describe what you hear (take notes for yourself).

Task 6 - Define two sine waves.

Along with $x[n]$, define a second sinewave, $x_2[n]$ that has a frequency, $f_0 = 7200\text{Hz}$, which is also sampled at $f_s = 8000\text{Hz}$.

Task 7 - Plot two sine waves.

Plot the two sinewaves. Alter the limits on the x-axis such that you only see a short portion of each signal. You can also listen to both.

Examine these plots. What do you observe? (Hint - think of what the frequency of each signal is, and compare to the sampling rate). Think about the results with reference to the theory you covered in lectures.

Task 8 - Exploring the Nyquist Theorem.

According to Nyquist, what is the minimum value of the sampling frequency that can be used to not degrade the signal above, $x(t)$?

You might like to take note of your answer, and check at the clinic that you were right.

To illustrate the Nyquist criterion, choose relevant values of f_s (choose multiple values above and below the minimum you identified) and resample the signal with $f_0 = 900\text{ Hz}$. You will need to redefine values for N , $t[n]$ as well as $x[n]$ for each new value of f_s you explore.

Task 9 - Listen at Different Sampling Rates.

Listen to the obtained numeric signals $x[n]$.

Can you verify the theory? We suggest you write down the values of f_s you chose in a table and explain what impact you have observed on the sound.

Task 10 - Exploring Aliasing using 'Handel' audio file.

Load the 'handel' audio .wav file in your script and listen to it.

What is the sampling rate of this signal? Now undersample at successively decreasing sampling frequencies until you notice a change. What is the frequency where you first perceive aliasing? Describe what the effect of aliasing sounds like to you. Plot the original audio file and its under-sampled version using subplot. As usual, label everything appropriately.

```
load handel.mat
filename = 'handel.wav';
audiowrite(filename,y,Fs);
clear y Fs
[y, Fs] = audioread('handel.wav');
```

2 Part 2: Musical Notes

We have considered so far a sine wave with fundamental frequency f_0 , with no concern about the significance of f_0 . In music, when a voice or instrument emits a particular frequency, that is described as a 'note'. For instance, on an 88 key piano, there are 88 unique 'notes' that can be played, each with their own fundamental frequency f_0 .

Each note in music can be assigned an integer value using a system known as MIDI. Here we can define as many as 128 unique notes expressed digitally as 7-bit binary numbers. Using this system we can compute the fundamental frequency of each note as follows:

$$f_0^{\text{note}} = 440 \times 2^{\frac{\text{note}-69}{12}} \quad (1)$$

In the above table, you can find some notes of the keyboard, their corresponding number and frequency.

In this part of the lab, we will create a new function called createNote which will take in a command from a main script and process it.

octave	C	C#	D	D#	E	F	F#	G	G#	A	A#	B
1	24 32.7	25 34.65	26 36.71	27 38.89	28 41.2	29 43.65	30 46.25	31 49	32 51.91	33 55	34 58.27	35 61.74
2	36 65.41	37 69.3	38 73.42	39 77.78	40 82.41	41 87.31	42 92.5	43 98	44 103.83	45 110	46 116.54	47 123.47
3	48 130.81	49 138.59	50 146.83	51 155.56	52 164.81	53 174.61	54 185	55 196	56 207.65	57 220	58 233.08	59 246.94
4	60 261.63	61 277.18	62 293.66	63 311.13	64 329.63	65 349.23	66 369.99	67 392	68 415.3	69 440	70 466.16	71 493.88
5	72 523.25	73 554.37	74 587.33	75 622.25	76 659.26	77 698.46	78 739.99	79 783.99	80 830.61	81 880	82 932.33	83 987.77
6	84 1046.5	85 1108.73	86 1174.66	87 1244.51	88 1318.51	89 1396.91	90 1479.98	91 1567.98	92 1661.22	93 1760	94 1864.66	95 1975.53
7	96 2093	97 2217.46	98 2349.32	99 2489.02	100 2637.02	101 2793.83	102 2959.96	103 3135.96	104 3322.44	105 3520	106 3729.31	107 3951.07

Figure 1: keyboard[1]

Task 1 - Create your scripts.

Begin this part of the lab by creating a main script called labAPart2.m and another new script called createNote.m.

Task 2 - Define note parameters.

Within the main script, define variables d , $note$, and f_s . $f_s = 8000$ Hz and the duration, d , and $note$ can have any values that you select.

Task 3 - Define createNote function input and outputs.

Define these parameters as inputs for the createNote.m function. This function will have two outputs defined as the signal, x and its corresponding time vector t .

Below is an example of how to define a function in a unique script in Matlab:

```
function [out1,out2] = functionName(in1,in2,in3)
```

Task 4 - Complete createNote function.

Now that the inputs and output of createNote.m are defined, complete the function by defining the equation for the note within the createNote function and also defining time vector t and the signal x .

Task 5 - Test createNote function.

Test that your function works by calling it in your main script and then listening to the generated output using the sound function.

What do you hear when comparing different notes? Describe it in your notes.

Using subplot, plot the signals for two different notes in the same figure. Ensure you correctly name the notes in the title of the Figure. Also include your Student ID in the title of the plot.

Include this plot in your notes and describe what the difference is between the waveforms of both notes.

Task 6 - Test createNote function with a special case.

Create a special case in your createNote.m function whereby if the note is equal to -1, x becomes a sequence of zeros. Within your main script, test if this works.

3 Part 3: Upsampling/Downsampling

It is often necessary to change the sampling rate of a discrete-time signal to obtain a new discrete-time representation of the underlying continuous-time signal of the form

$$x_1[n] = x_c(nT_1)$$

where the original sampling period was T and $T \neq T_1$. This operation is called resampling.

Task 1 - Upsampling Sine Wave.

Begin this part of the lab by creating a main script called labAPart3.m and perform the following tasks within this script. Pick a note/frequency from Figure 1. Design the pure sine wave of that frequency, by selecting the sampling frequency and the duration. Upsample your signal by a factor of 2 by first making every second sample equal to 0 (expansion), and then make the missing samples be a simple average of the samples either side (interpolation).

Plot both your signal and the upsampled version. Examine the plot using appropriate xlim values to show the difference between the two. How does upsampling affect your signal? Try different sampling factors and more advanced interpolation.

Task 2 - Upsampling Music.

Reload the 'handel' wav file. Upsample it by an integer factor of 2, and then 4. Listen to the output. Repeat this upsampling process, until you perceive artefacts in the music.

Plot the original and the downsampled signals. Listen to the original and distorted upsampled signal. At what level of upsampling do you begin to perceive distortions? What is the effective sampling frequency? How does this relate to the theory you covered in lectures? Save the resulting .wav file.

```
audiowrite('Local_folder/sample_audio.wav', y, Fs); % Write audio to a local file
```

Caution: The resample built-in function in MATLAB uses an antialiasing lowpass filter. Since the goal of this question is to observe aliasing, you must upsample the signal without any anti-aliasing filter, i.e. Task 1 and 2 will not give you the intended experience of aliasing if you use resample. You need to first expand the signal - by putting zero-valued samples between actual samples to increase the sampling rate. Then you need to consider how to implement the interpolation part. Refer to your notes from Lecture 5.

4 Part 4: Extra Optional Challenge: Creating a Melody

If you found the material to this point easy and want a challenge, then try out this final part.

In music, a melody is defined as a sequence of notes.

In this part of the lab, you will create a new function called `createMelody.m` which will call upon the `createNote.m` function from part 2 of the lab to create a melody.

Task 1 - Create your scripts.

Create two new MATLAB scripts entitled `labApart4.m` and `createMelody.m`.

Task 2 - Define note parameters.

As with part 2, you will need to define the duration and value of the notes in your melody. The sampling frequency f_s will remain constant. In your main script, define vectors of note values and note duration's. These will determine how the melody will sound.

Task 3 - Define `createMelody.m` function inputs and outputs.

Your `createMelody.m` function will have 3 inputs consisting of the note value vector, the note duration vector, and the single value sampling rate, f_s . The function will output the signal, x .

Task 4 - Complete `createMelody.m` function.

Now that the inputs and output of `createMelody.m` are defined, write the function. The simplest way to create this function is to create an empty vector x . Use a for loop to go through each note in the melody and use the `createNote.m` function from part 2 to create this note. Make a temporary signal tmp within the for loop to save the signal from the `createNote.m` function. Update the vector x by concatenating the signal from tmp onto the vector x .

Task 4 - Test `createMelody.m` function.

Test your function in the script `labAPart4.m` by selecting vectors d_{vect} and $note_{vect}$ of your choice. Listen to the melody. Save it as a wav. file.

```
audiowrite('Local_folder/sample_audio.wav', y, Fs); % Write audio to a local file
```

Plot the signal against time. Clearly indicate the notes and duration that you have chosen.

Task 5 - Modify Your Melody.

Please modify this melody so its height is one octave up. An octave is a gap of 12 notes, between two

notes. For instance, an octave separates the note 45 (A octave 2) from the note 57 (A octave 3).

Explain how you do it. Please make the melody go 50% faster. Explain what you need to change. Save it as another wav. file, with a suitably informative name.

Task 6 - Create a Random Melody.

Create a random melody of notes with random durations and values. Also randomly select how many notes will be in the sequence. Choose reasonable ranges for these note values. *Test your function, and save the created music in a wav. file. Comment on how melodic your music is (or is not!).*

```
randomNote = randi([24, 107]); %randomly selects an integer value between 24 and  
107
```

5 Required submission

The purpose of this lab is that you engage with the assigned tasks, get a better appreciation of the real-world analysis of signals, and make connections between what you are learning in lectures and practical use of those concepts. The purpose of the submission is to show you have engaged, and to establish a record of the code and material that you are permitted to use later in the module for your Class MATLAB Assignment. By the time you are complete, you should have done all the assigned tasks, thought about the questions asked, and be achieving the required level of proficiency in MATLAB.

With this in mind, you must submit your code. All code must be adequately commented and we should be able to run your code to recreate the required plots etc. You can optionally submit your own notes/record of the lab.

Note: All deadlines are detailed on Blackboard, and students must make themselves aware of when items are due. No automatic extensions can be given as this material is aligned for module delivery.

References

- [1] Fundamental frequencies of notes in western music. <https://auditoryneuroscience.com/pitch/fundamental-frequencies-notes-western-music>.